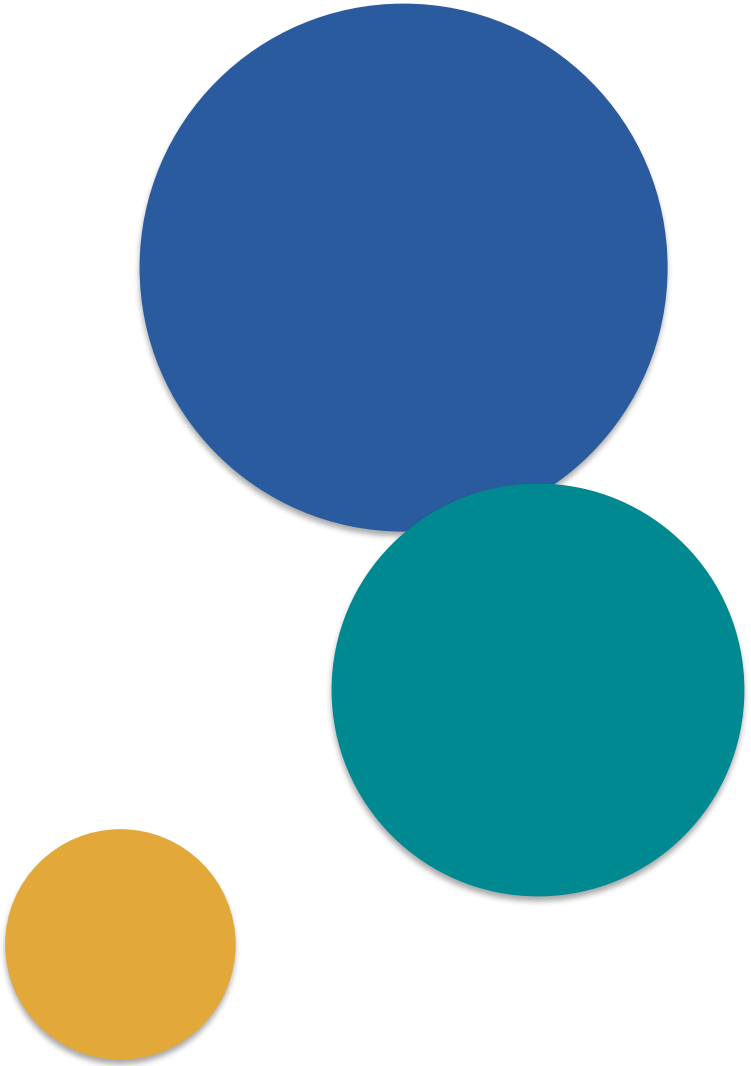
Introduction to Python for Engineers

Engineering Information Systems and Digital Skills

Lecture 10 | First-year Engineering Students

Concepts, demonstrations and home practice



By the end of this lecture, you should be able to...

Learning outcomes

- Explain what Python is and why engineers use it.
- Recognize where Python code is written and run.
- Understand print(), variables, basic data types and arithmetic.
- Use input(), conditions, loops and lists in simple examples.
- Read a short engineering-related Python program.
- Identify common errors and plan independent practice.

Why learn Python?

Python is one useful tool—not the only language engineers need.

Python is approachable

- Readable syntax that is close to everyday language.
- Useful for learning programming ideas.
- Large community and many learning resources.
- Works on common operating systems.

Python is useful in engineering

- Analyze measurement data.
- Automate repetitive calculations.
- Explore mathematical models.
- Work with sensors, AI and scientific tools.

What is Python?

Today: understand the building blocks, not advanced programming.

A programming language

Python lets people write instructions that a computer can execute.

A program is a sequence of instructions written to complete a task.

An example task

Read a temperature measurement.

Convert Celsius to Fahrenheit.

Display the result.

A program can repeat this task quickly and consistently.

From problem to result

Programming involves testing and improving—not only writing code.

1 DEFINE

What do you want the computer to do?

2 WRITE CODE

Express the steps using Python instructions.

3 RUN

The Python environment executes the program.

4 CHECK

Compare the output with what you expected.

Where can you write and run Python?

The exact tool may differ; the basic Python ideas stay the same.

Code editor or IDE

An editor helps you write and save code.

An IDE adds tools such as running, debugging and project management.

Examples: IDLE, VS Code and PyCharm.

Notebook or online environment

A notebook lets you run small pieces of code and view results step by step.

Online environments can reduce setup time, but require a suitable internet connection.

Your first Python program

A command that displays a message

Code

```
print("Hello, engineering!")
```

Output

```
Hello, engineering!
```

print() displays text or a value in the output area.

Reading Python syntax

Start with short programs that do one clear task.

Punctuation matters

- Text messages are written inside quotation marks.
- Parentheses contain the information passed to print().
- Spelling and brackets must match.

Python is case-sensitive

print() and Print() are not the same name.

Example:

print("Ready")

Use consistent spelling and indentation.

Comments: notes for people

Code

```
# Convert a temperature
print(25)
```

What happens?

The comment begins with #.

Python ignores the comment when running the program.

The program displays 25.

Comments explain the purpose of code and make it easier to understand later.

Variables: giving values a name

Code

```
temperature = 25
limit = 30
print(temperature)
print(limit)
```

Think of a variable as a labelled box

temperature stores 25

limit stores 30

The = symbol assigns a value to a name.

Good variable names

Use letters, numbers and underscores; do not start a name with a number.

Clear names

temperature = 25

sensor_reading = 18.6

motor_speed = 1200

Names should describe what a value means.

Avoid unclear names

x = 25

v = 18.6

n = 1200

Short names can be acceptable in mathematics, but descriptive names help when programs grow.

Basic data types

A value's type affects what operations make sense.

INTEGER (int)

Whole numbers

Example: 12

DECIMAL (float)

Numbers with a decimal part

Example: 18.6

TEXT (str)

Characters inside quotes

Example: "Sensor A"

BOOLEAN (bool)

True or False

Useful for conditions.

Basic arithmetic in Python

Python can perform calculations directly

Code

```
a = 12
b = 5
print(a + b)
print(a - b)
print(a * b)
print(a / b)
```

Results

17 → addition

7 → subtraction

60 → multiplication

2.4 → division

Useful arithmetic operators

Two more to recognize: ** means power (2 ** 3 gives 8); % gives the remainder (10 % 3 gives 1).

+ ADD

a + b

- SUBTRACT

a - b

* MULTIPLY

a * b

/ DIVIDE

a / b

Order of operations

Use parentheses when they make the intended calculation clearer.

Parentheses help

result = (2 + 3) * 4

The addition happens first.

The result is 20.

Without parentheses

result = 2 + 3 * 4

Multiplication happens first.

The result is 14.

Getting input from the user

Code

```
name = input("Enter your name: ")
print("Hello", name)
```

What happens?

1. Python displays the prompt.
2. The user types a response.
3. input() returns the response as text.
4. print() displays a greeting.

Numbers entered by a user

Convert text input before doing arithmetic

Code

```
value = float(input("Enter temperature: "))
print(value + 5)
```

Why float()?

input() returns text.

float() converts suitable numeric text into a decimal number.

Use int() for whole-number input when appropriate.

Worked example: Celsius to Fahrenheit

Code

```
celsius = 25
fahrenheit = (celsius * 9 / 5) + 32
print(fahrenheit)
```

Follow the calculation

Start: 25 °C

Multiply by 9/5: 45

Add 32: 77

Output: 77.0

The formula is applied consistently each time.

Making decisions with if

Code

```
temperature = 35
if temperature > 30:
    print("Check cooling")
```

Read it carefully

if checks whether a condition is true.

The indented line runs only when temperature > 30 is true.

The colon and indentation are important.

if, elif and else

Conditions commonly use >, <, >=, <=, == and !=.

Choose between conditions

```
if temperature > 30:
    print("High")
elif temperature < 10:
    print("Low")
```

Handle the other cases

```
else:
    print("Moderate")
```

elif checks another condition if the earlier one was false.
else handles the remaining cases.

Comparison operators

A comparison produces True or False. One equals sign (=) assigns; two (==) compare.

> GREATER THAN

temperature > 30

< LESS THAN

speed < 100

== EQUAL TO

status == 1

!= NOT EQUAL TO

mode != 0

Loops: repeat a task

Code

```
for reading in range(3):
    print("Take a reading")
```

Output

Take a reading

Take a reading

Take a reading

The loop repeats the indented instructions three times.

Understanding range()

Indentation shows which instructions belong to the loop.

Example

```
for i in range(5):
    print(i)
```

Output:

0

1

2

3

4

Important point

range(5) generates five values starting from 0 and stopping before 5.

Loops help process repeated measurements or perform repeated calculations.

Lists: storing several measurements

Code

```
temperatures = [21.5, 22.0, 22.8]
print(temperatures[0])
print(len(temperatures))
```

What do the lines mean?

The list stores three values.

Index 0 means the first item, so temperatures[0] is 21.5.

len() returns the number of items: 3.

Processing each measurement

Code

```
temperatures = [21.5, 22.0, 22.8]
for t in temperatures:
    print(t)
```

Why this is useful

The loop takes each value from the list in turn.

The same pattern can be used for sensor readings, test results or measured dimensions.

Functions: reusable instructions

Code

```
def c_to_f(celsius):
    return (celsius * 9 / 5) + 32

print(c_to_f(25))
```

Key ideas

def starts a function definition.

celsius is the input parameter.

return sends the result back.

The function can be used for many values.

Common beginner errors

Read the error message, check the line, and test with a small example.

SYNTAX ERROR

Missing colon, bracket or quotation mark.

NAME ERROR

Using a variable name that has not been defined.

TYPE ERROR

Trying an operation that does not suit a value's type.

LOGIC ERROR

The program runs, but the method or formula is wrong.

How to debug a small program

A program that runs is not automatically a correct program.

A simple checking routine

1. Read the error message.
2. Find the line mentioned.
3. Check spelling, brackets, colons and indentation.
4. Test one small part at a time.

Check the result

Try a value whose answer you can calculate by hand.

For engineering work, also check units, assumptions and the formula—not just the Python syntax.

Mini engineering example: classify a temperature

Code

```
temperature = 35
if temperature > 30:
    print("Above target range")
else:
    print("Within target range")
```

Discuss

What value is stored?

Which condition is checked?

Which message is displayed for 35?

What changes if temperature = 25?

30-minute interaction & practice

Use an editor or notebook available to you.

Activity 1 — predict the output (10 min)

Without running it, predict the output:

```
x = 4
print(x + 3)
print(x * 2)
```

Then run the code and compare.

Activity 2 — engineering task (20 min)

Write a short program that stores three sensor readings in a list and prints each reading.

Extension: calculate and display the average.

Work in pairs and explain each line.

Home practice: small steps

Bring one program and one question to the next session.

Key points

- Write a program that displays your name and engineering programme.
- Store a length in metres and display it in centimetres.
- Ask the user for a temperature and convert it to Fahrenheit.
- Use if/else to classify a measurement as above or below a chosen limit.
- Create a list of three measurements and print each one using a loop.

Lecture 10 Review: Multiple Choice — 1.1

Choose the best answer. Check the answer key only after all questions.

Question 1

What does print() do?

A. Stores a file permanently

B. Displays information in the output

C. Repeats a loop

D. Defines a function

Question 2

Which line assigns the value 25 to a variable named temperature?

A. temperature == 25

B. 25 = temperature

C. temperature = 25

D. print temperature 25

Lecture 10 Review: Multiple Choice — 1.2

Choose the best answer. Check the answer key only after all questions.

Question 3

Which data type is most suitable for the value 18.6?

A. float

B. bool

C. str only

D. int only

Question 4

What does input() normally return?

A. A list

B. A decimal number automatically

C. A Boolean value

D. Text (a string)

Lecture 10 Review: Multiple Choice — 2.1

Choose the best answer. Check the answer key only after all questions.

Question 5

What does == test?

A. Assigns a value

B. Compares whether values are equal

C. Starts a loop

D. Converts text to a number

Question 6

Which keyword is used to make a decision based on a condition?

A. for

B. def

C. if

D. return

Lecture 10 Review: Multiple Choice — 2.2

Choose the best answer. Check the answer key only after all questions.

Question 7

What values are produced by range(3) in a simple for loop?

A. 1, 2, 3

B. 0, 1, 2

C. 0, 1, 2, 3

D. 3 only

Question 8

What is the index of the first item in a Python list?

A. 0

B. 1

C. -1 only

D. The list length

Lecture 10 Review: Multiple Choice — 3.1

Choose the best answer. Check the answer key only after all questions.

Question 9

Which structure is useful for processing several sensor readings?

A. A comment

B. A quotation mark

C. A list and a loop

D. A single print statement only

Question 10

What is a logic error?

A. A missing closing bracket

B. A misspelled variable name that stops execution

C. An unsupported data type operation

D. A program runs but gives the wrong result

Lecture 10 Review: Multiple Choice — 3.2

Choose the best answer. Check the answer key only after all questions.

Question 11

What does float() do with suitable numeric text such as '3.5'?

A. Converts it to a decimal number

B. Prints it without conversion

C. Creates a loop

D. Converts it to True

Question 12

What does return do in a function?

A. Ends the entire computer session

B. Sends a result back to the caller

C. Repeats the function forever

D. Converts all values to text

Lecture 10 Review: Answer Key

Check your answers after completing all 12 questions.

Questions 1–4

1. B — Displays information
2. C — temperature = 25
3. A — float
4. D — input returns text

Questions 5–8

5. B — Compares equality
6. C — if
7. B — 0, 1, 2
8. A — Index 0

Questions 9–12

9. C — List and loop
10. D — Runs but wrong result
11. A — Converts to float
12. B — Sends back a result

Lecture summary

Start small, run the code, inspect the result, and improve it.

Key points

- Python is a readable language used for automation, data analysis and engineering calculations.
- Variables store values; basic types include int, float, str and bool.
- Arithmetic operators perform calculations; input() reads user responses.
- if/elif/else selects actions based on conditions.
- Loops repeat instructions; lists store several values; functions organize reusable steps.
- Debugging means finding and fixing errors—and checking whether results make sense.

Exit ticket & next steps

One-minute individual reflection

Before you leave

Explain in your own words:

1. What is a variable?
2. What is the difference between = and ==?
3. When would you use a loop?

Next step: independent practice

Run the home-practice examples.

Change one value and predict what will happen.

Keep your code and notes for the formal programming course.